

remote-operations 설계·운영 매뉴얼

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

1. 책임 범위

remote-operations 는 management host의 remote-boot.service 가 실행하는 부팅 orchestration이다. Wake-on-LAN, 우선 서버 gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 monitoring 에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 remote-boot.service 하나다.

2. 디렉터리 지도

경로	핵심 기능
script/run_remote_boot.sh	전체 부팅 흐름의 entry point
script/wake_targets.sh	target 선택과 WOL magic packet 전송
script/wait_for_priority_servers.sh	priority/remaining host health gate
script/check_server_boot_health.sh	mount, GPU, 임시 test container 점검
script/restart_all_remote_containers.sh	기존 container 시작과 SSH/GPU post-check
script/create_test_container.sh	health check용 임시 GPU container 생성
script/delete_test_container.sh	임시 container 정리
script/common.sh	config, target, Ansible, log, alert 공통 함수
script/install_remote_boot_service.sh	systemd unit 설치·활성화
script/dry_run_remote_boot.sh	WOL/health/container/full flow simulation
script/integration_smoke_test.sh	수동 Ansible/Docker/GPU 통합 확인
config/remote_boot.example.env	공개 가능한 설정 구조
config/remote_boot.local.env	실제 MAC·webhook 등을 담은 ignored 설정

3. 부팅 상태 머신

```
pre-delay
|
v
wake priority targets
|
v
priority health gate ---- failure -----> stop + alert
|
v
secondary delay
|
v
wake remaining targets
|
v
remaining health gate -- failure -----> record + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

각 domain에서 서버 한 대를 먼저 깨우는 구체적인 이유는 **LAB1** 과 **FARM1** 에 사용자 관리 DB가 Docker container로 존재하기 때문이다. 이 DB는 사용자, UID/GID, 할당 port와 사용자 container record를 관리하는 기준 데이터이므로, 나머지 compute server보다 먼저 DB host를 기동해 관리 상태가 복구될 시간을 확보한다.

따라서 **LAB1** 과 **FARM1** 을 priority target으로 먼저 깨우고 health gate를 통과한 뒤 나머지 서버를 기동한다. 이렇게 하면 사용자 관리 DB가 준비되지 않은 상태에서 전체 서버와 사용자 container 운영이 동시에 시작되는 상황을 줄일 수 있다. priority gate를 끌 수는 있지만 기본값은 이 의존 순서를 보존한다.

4. 핵심 기능

4.1 target 정규화

설정은 FARM/LAB 전체 목록, 기본 대상과 priority 대상을 별도로 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. **all**, domain group, 개별 **FARM1** / **LAB10** 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

4.2 1회 host health check

`check_server_boot_health.sh` 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount
- host `nvidia-smi`
- 임시 test container 생성
- container 내부 GPU와 필요한 home mount
- test 종료 후 container 삭제

실제 사용자 container 대신 고정된 health test identity와 image를 사용하는 이유는 특정 사용자 상태를 서버 health로 오판하지 않고, test artifact를 명확히 정리하기 위해서다.

4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle`에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh` 는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

5. 설정 모델

설정은 `remote_boot.example.env` 를 복사하여 local 파일에서 관리한다.

```
cd /home/jy/server_manage/remote-operations
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

그룹	예
target	FARM/LAB 목록, 기본/priority target
순서/gate	pre-delay, gate timeout/poll, secondary delay

그룹	예
container	restart enable, timeout, post-check poll
network	Ansible inventory, broadcast IP, MAC address
health	필수 mount와 host share template
test container	image, UID/GID, mount, memory, runtime
logging/alert	log path, rotate count, notify API와 webhook

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

6. 사용법

target 확인과 수동 WOL:

```
cd /home/jy/server_manage/remote-operations
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh
sudo systemctl start remote-boot.service
systemctl status remote-boot.service
journalctl -u remote-boot.service -b
```

7. dry-run과 검증

```
./script/dry_run_remote_boot.sh wake FARM1 LAB1
./script/dry_run_remote_boot.sh health FARM1
./script/dry_run_remote_boot.sh containers FARM1
./script/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./script/integration_smoke_test.sh --scope priority
```

운영 전에는 최소한 다음을 확인한다.

- priority target이 실제 의존 순서와 맞는지
- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- health test image가 target GPU driver와 호환되는지
- 임시 container 이름이 실제 사용자 container와 충돌하지 않는지
- gate timeout이 정상 부팅 시간보다 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

8. 실패 처리 원칙

- priority gate 실패 시 나머지 서버를 깨우지 않는 것이 기본이다.
- remaining server 일부 실패는 성공한 서버의 후속 작업과 실패 대상을 구분해 기록한다.
- test container는 성공/실패와 무관하게 정리를 시도한다.
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 **monitoring**으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

9. 새 서버 추가

1. `user-lifecycle/server_info/servers.jsonl` 과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 priority 여부와 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.

6. container restart를 한 서버에 제한해 검증한다.
7. `monitoring`의 scrape target/exporter 배포를 별도로 완료한다.

10. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.
- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.